Name of Student:	PRN:
Semester: III	Year: AY 2026-27
Subject: Programming with Java	
ASSIGNMENT No: 28	
TITLE: Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication	

Problem Statement

Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives

To implement Remote Method Invocation for distributed Java applications.

Programme

1. Create a remote calculator service using Java RMI

1:CalculatorClient.java:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class CalculatorClient {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.getRegistry("localhost", 1099);

        Calculator calculator = (Calculator) registry.lookup("CalculatorService");

        Scanner sc = new Scanner(System.in);
```

```
        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        System.out.println("Addition: " + calculator.add(a, b));
        System.out.println("Subtraction: " + calculator.subtract(a, b));
        System.out.println("Multiplication: " + calculator.multiply(a, b));

        if (b != 0) {
            System.out.println("Division: " + calculator.divide(a, b));
        } else {
            System.out.println("Division: Cannot divide by zero");
        }

        sc.close();
    }
}
```

2 : CalculatorServer.java:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class CalculatorServer {
```

```
    public static void main(String[] args) throws Exception {
        CalculatorImpl calculator = new CalculatorImpl();

        Registry registry = LocateRegistry.createRegistry(1099);
        registry.rebind("CalculatorService", calculator);

        System.out.println("Calculator Server is running");
    }
}
```

3:CalculatorImpl.java:

```
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;

public class CalculatorImpl extends UnicastRemoteObject implements Calculator
{
    public CalculatorImpl() throws RemoteException {
        super();
    }

    public int add(int a, int b) {
        return a + b;
    }

    public int subtract(int a, int b) {
        return a - b;
```

```
    }

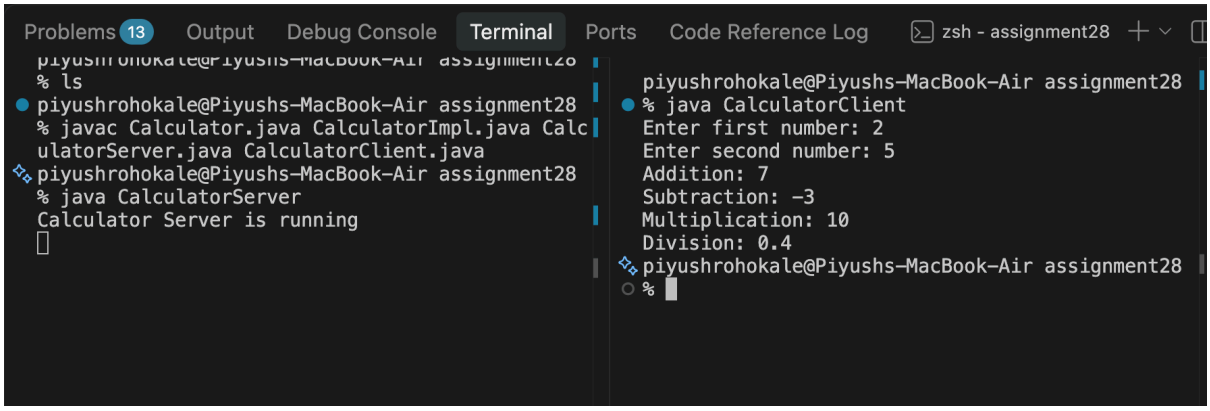
    public int multiply(int a, int b) {
        return a * b;
    }

    public double divide(int a, int b) throws RemoteException {
        if (b == 0) {
            throw new RemoteException("Cannot divide by zero");
        }
        return (double) a / b;
    }
}
```

4: Calculator.java:

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Calculator extends Remote {
    int add(int a, int b) throws RemoteException;
    int subtract(int a, int b) throws RemoteException;
    int multiply(int a, int b) throws RemoteException;
    double divide(int a, int b) throws RemoteException;
}
```



2. Create a Java application using RMI where the server converts temperature values between Celsius and Fahrenheit. The client sends input temperature and receives converted output.

1: Temperature.java:

```
import java.rmi.Remote;

import java.rmi.RemoteException;


public interface Temperature extends Remote {

    double celsiusToFahrenheit(double celsius) throws RemoteException;

    double fahrenheitToCelsius(double fahrenheit) throws RemoteException;

}
```

2: TemperatureImpl.java:

```
import java.rmi.RemoteException;

import java.rmi.server.UnicastRemoteObject;


public class TemperatureImpl extends UnicastRemoteObject implements Temperature {

    public TemperatureImpl() throws RemoteException {
```

```
        super();
    }

    public double celsiusToFahrenheit(double celsius) {
        return (celsius * 9 / 5) + 32;
    }

    public double fahrenheitToCelsius(double fahrenheit) {
        return (fahrenheit - 32) * 5 / 9;
    }
}
```

3: TemperatureServer.java:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;

public class TemperatureServer {
    public static void main(String[] args) throws Exception {
        TemperatureImpl temperature = new TemperatureImpl();

        Registry registry = LocateRegistry.createRegistry(1099);
        registry.rebind("TemperatureService", temperature);

        System.out.println("Temperature Server is running");
    }
}
```

4: TemperatureClient.java:

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;

public class TemperatureClient {
    public static void main(String[] args) throws Exception {
        Registry registry = LocateRegistry.getRegistry("localhost", 1099);

                        Temperature temperature = (Temperature)
registry.lookup("TemperatureService");

        Scanner sc = new Scanner(System.in);

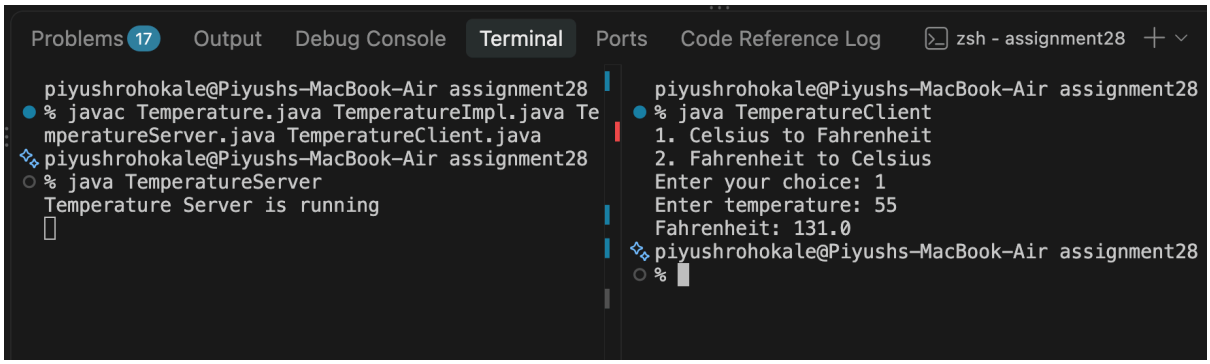
        System.out.println("1. Celsius to Fahrenheit");
        System.out.println("2. Fahrenheit to Celsius");
        System.out.print("Enter your choice: ");
        int choice = sc.nextInt();

        System.out.print("Enter temperature: ");
        double value = sc.nextDouble();

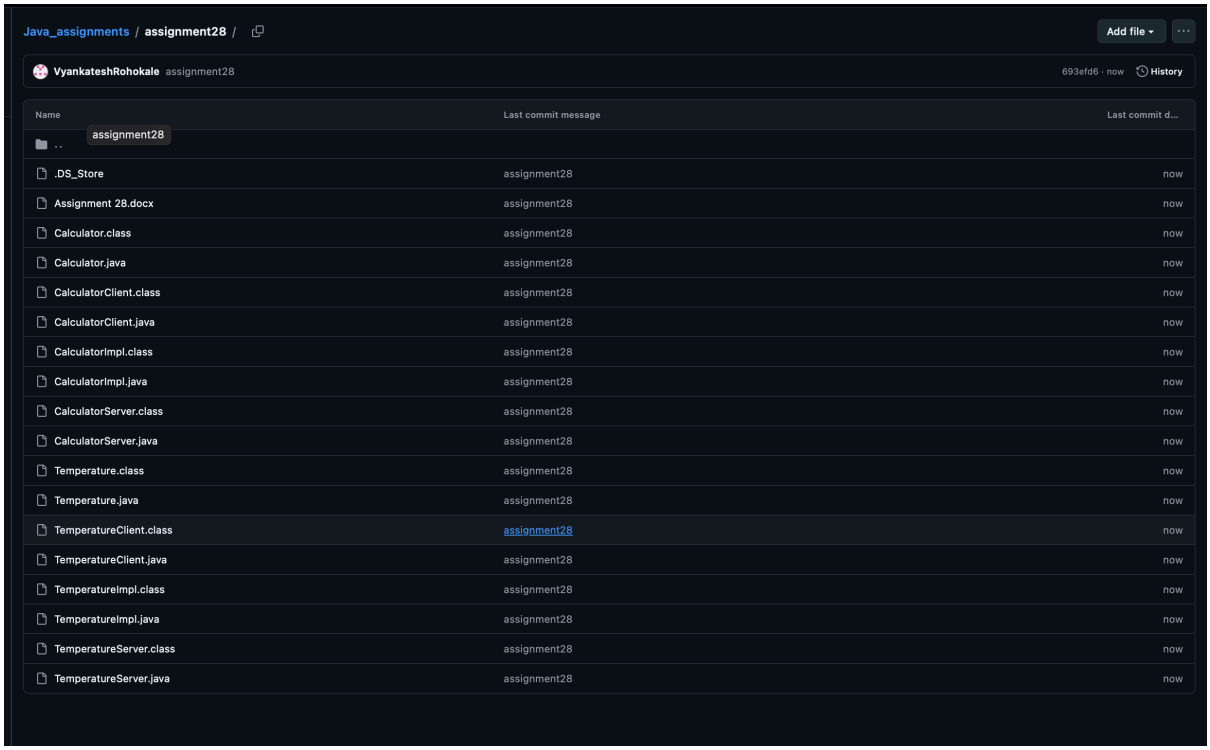
        if (choice == 1) {
                                    System.out.println("Fahrenheit: " +
temperature.celsiusToFahrenheit(value));
```

```
        } else if (choice == 2) {
            System.out.println("Celsius: " + temperature.fahrenheitToCelsius(value));
        } else {
            System.out.println("Invalid choice");
        }

        sc.close();
    }
}
```



Github :



Conclusion:

we implemented a temperature conversion application using Java RMI. The client sends the temperature and conversion request to the server, which performs the calculation and returns the converted value.